
CS771 Minor Assignment 2:

Melbo's Mesmerizing Mixtures

Detailed Derivations for GMM Generative Classifier

Yash Pathak
Roll No. 231188
Dept Civil Eng, IIT Kanpur
yashp@iitk.ac.in

Sanyam Jain
Roll No. 230924
Dept of Civil Eng, IIT Kanpur
sanyamjain23@iitk.ac.in

Shreyas Chougankar
Roll No. 230990
BS MTH, IIT Kanpur
shreyasc23@iitk.ac.in

Priyanshu Mishra
Roll No. 230806
Dept of Civil Eng, IIT Kanpur
priyanshu23@iitk.ac.in

Vaibhav Gupta
Roll No. 231114
Dept of Civil Eng, IIT Kanpur
gvaibhav23@iitk.ac.in

Vishesh Thepadia
Roll No. 231165
BS Physics, IIT Kanpur
vishesht23@iitk.ac.in

Problem 2.1: Melbo's Mesmerizing Mixtures

This report provides a detailed derivation of the training and inference procedures for a generative multi-class classifier. This model extends the single-Gaussian-per-class model by using a full Gaussian Mixture Model (GMM) for each class-conditional density, $\mathbb{P}[x|y = c]$. We also incorporate regularization to improve numerical stability and prevent model degeneracy.

Part 1: Detailed Calculations for Training (with Regularization)

Our goal is to learn the parameters Θ of our generative model from a training dataset $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$. The model is defined by:

1. **Class Priors:** $\mathbb{P}[y = c] = \phi_c$.
2. **Class-Conditionals:** $\mathbb{P}[x|y = c] = \sum_{k=1}^K \pi_k^c \mathcal{N}(x|\mu_k^c, \Sigma_k^c)$.

The complete set of parameters to be learned is $\Theta = \{\{\phi_c\}_{c=1}^C, \{\pi_k^c, \mu_k^c, \Sigma_k^c\}_{k=1}^K\}_{c=1}^C$.

We use the **Maximum Likelihood Estimation (MLE)** framework, which seeks to find parameters Θ that maximize the log-likelihood of the observed data. The total log-likelihood is:

$$\mathcal{L}(\Theta) = \sum_{i=1}^N \log \mathbb{P}[x^{(i)}, y^{(i)}|\Theta]$$

Using the chain rule, $\mathbb{P}[x, y] = \mathbb{P}[x|y]\mathbb{P}[y]$, we can split the log-likelihood:

$$\mathcal{L}(\Theta) = \sum_{i=1}^N \left(\log \mathbb{P}[x^{(i)}|y^{(i)}, \Theta] + \log \mathbb{P}[y^{(i)}|\Theta] \right)$$

This conveniently decouples the optimization. We can maximize the terms for the class priors and the class-conditionals independently.

Step 1: Estimating Class Priors (with Regularization)

We first optimize the class priors, $\{\phi_c\}$. The relevant part of the log-likelihood is:

$$\mathcal{L}_{\text{priors}}(\{\phi_c\}) = \sum_{i=1}^N \log \mathbb{P}[y^{(i)}|\{\phi_c\}] = \sum_{c=1}^C N_c \log(\phi_c)$$

where $N_c = \sum_{i=1}^N \mathbb{I}(y^{(i)} = c)$ is the number of samples in class c .

Problem with MLE: A standard MLE solution would be $\phi_c = N_c/N$. However, if any class c has zero samples in the training set ($N_c = 0$), its prior would be $\phi_c = 0$. This leads to $\log(\phi_c) = -\infty$, which will permanently prevent this class from ever being predicted.

Solution (MAP with Dirichlet Prior): To solve this, we use **Laplace (or add- α) smoothing**. This is equivalent to finding the Maximum A Posteriori (MAP) estimate assuming a Dirichlet prior on the vector $\vec{\phi}$. With a smoothing parameter $\alpha > 0$ (e.g., $\alpha = 1$):

$$\phi_c = \frac{N_c + \alpha}{N + C \cdot \alpha}$$

This ensures that ϕ_c is always positive, preventing numerical errors.

Step 2: Estimating Class-Conditional GMM Parameters

For each class c , we must optimize its GMM parameters $\Theta_c = \{\pi_k^c, \mu_k^c, \Sigma_k^c\}_{k=1}^K$ by maximizing the class-conditional log-likelihood for the data in that class, $D_c = \{x^{(i)} | y^{(i)} = c\}$:

$$\mathcal{L}(\Theta_c) = \sum_{x \in D_c} \log \mathbb{P}[x | \Theta_c] = \sum_{x \in D_c} \log \left(\sum_{k=1}^K \pi_k^c \mathcal{N}(x | \mu_k^c, \Sigma_k^c) \right)$$

Problem (Intractable Log-Likelihood): This optimization is analytically intractable. The log of a sum does not simplify, and taking derivatives to find a closed-form solution is impossible.

Solution (Expectation-Maximization): We use the **Expectation-Maximization (EM) Algorithm**. EM is an iterative algorithm that solves this problem by introducing latent variables z . Let $z^{(i)}$ be a latent variable for $x^{(i)}$ indicating which component k generated it.

The EM algorithm iterates two steps:

- **E-Step:** Compute the **expected** log-likelihood of the "complete" data (with z) given the **observed** data x and current parameters Θ_{old} . This expectation is called the Q -function.
- **M-Step:** Maximize this Q -function to find the new parameters Θ_{new} .

E-Step (Expectation): We compute the "responsibility" γ_{ik}^c for each $x^{(i)} \in D_c$. This value, $\gamma_{ik}^c = \mathbb{P}[z^{(i)} = k | x^{(i)}, \Theta_c]$, represents our belief that component k was responsible for generating point $x^{(i)}$. By Bayes' rule:

$$\gamma_{ik}^c = \frac{\mathbb{P}[z^{(i)} = k] \mathbb{P}[x^{(i)} | z^{(i)} = k]}{\mathbb{P}[x^{(i)}]} = \frac{\pi_k^c \mathcal{N}(x^{(i)} | \mu_k^c, \Sigma_k^c)}{\sum_{j=1}^K \pi_j^c \mathcal{N}(x^{(i)} | \mu_j^c, \Sigma_j^c)}$$

We compute this for all $i \in D_c$ and $k \in [K]$.

M-Step (Maximization) with Regularization: We update the parameters Θ_c by maximizing the expected complete-data log-likelihood, which simplifies to maximizing the following weighted log-likelihood:

$$\Theta_c^{\text{new}} = \arg \max_{\Theta_c} \sum_{i=1}^{N_c} \sum_{k=1}^K \gamma_{ik}^c \log(\pi_k^c \mathcal{N}(x^{(i)} | \mu_k^c, \Sigma_k^c))$$

This larger optimization problem conveniently decouples for π , μ , and Σ .

1. **Update Mixture Weights π_k^c :** We maximize $\sum_{i=1}^{N_c} \sum_{k=1}^K \gamma_{ik}^c \log(\pi_k^c)$ subject to $\sum_{k=1}^K \pi_k^c = 1$. Using a Lagrange multiplier, the solution is:

$$\pi_k^c = \frac{\sum_{i=1}^{N_c} \gamma_{ik}^c}{N_c} = \frac{N_k^c}{N_c}$$

where $N_k^c = \sum_{i=1}^{N_c} \gamma_{ik}^c$ is the "effective" number of points assigned to component k .

2. **Update Component Means μ_k^c :** We maximize the part of the likelihood involving μ_k^c , which is equivalent to minimizing a weighted least-squares error:

$$\mu_k^c = \arg \min_{\mu} \sum_{i=1}^{N_c} \gamma_{ik}^c (x^{(i)} - \mu)^T (\Sigma_k^c)^{-1} (x^{(i)} - \mu)$$

Taking the derivative with respect to μ_k^c and setting to zero yields the weighted mean:

$$\mu_k^c = \frac{\sum_{i=1}^{N_c} \gamma_{ik}^c x^{(i)}}{\sum_{i=1}^{N_c} \gamma_{ik}^c} = \frac{1}{N_k^c} \sum_{i=1}^{N_c} \gamma_{ik}^c x^{(i)}$$

3. **Update Component Covariances Σ_k^c (with Regularization):** The unregularized update for the covariance is the weighted sample covariance:

$$\Sigma_k^c(\text{unreg}) = \frac{1}{N_k^c} \sum_{i=1}^{N_c} \gamma_{ik}^c (x^{(i)} - \mu_k^c)(x^{(i)} - \mu_k^c)^T$$

Problem: This matrix can become **singular** (non-invertible) if N_k^c is small or if the points $x^{(i)}$ assigned to component k are collinear (e.g., all lie on a line). This would make $\log |\Sigma_k^c| = -\infty$ and crash the algorithm.

Solution: We add a small, positive diagonal matrix ϵI (where $\epsilon \sim 10^{-6}$ and I is the identity matrix). This **regularization** ensures the covariance matrix is always positive definite and invertible.

$$\Sigma_k^c = \left(\frac{1}{N_k^c} \sum_{i=1}^{N_c} \gamma_{ik}^c (x^{(i)} - \mu_k^c)(x^{(i)} - \mu_k^c)^T \right) + \epsilon I$$

We iterate the E-step and M-step for each class c until the log-likelihood $\mathcal{L}(\Theta_c)$ converges.

Part 2: Detailed Calculations for Inference (with Log-Sum-Exp Trick)

For a new test point x_{test} , we want to find the most probable class $\hat{y}$. We use the **Maximum A Posteriori (MAP)** rule:

$$\hat{y} = \arg \max_{c \in [C]} \mathbb{P}[y = c | x_{test}]$$

Using Bayes' theorem:

$$\mathbb{P}[y = c | x_{test}] = \frac{\mathbb{P}[x_{test} | y = c] \mathbb{P}[y = c]}{\mathbb{P}[x_{test}]}$$

Since $\mathbb{P}[x_{test}]$ is a constant normalizer across all classes, it does not affect the arg max:

$$\hat{y} = \arg \max_{c \in [C]} \mathbb{P}[x_{test} | y = c] \mathbb{P}[y = c]$$

Problem (Numerical Underflow): Probabilities are small numbers. Multiplying them (e.g., $\pi_k^c \cdot \mathcal{N}(\dots)$) can result in numbers smaller than the machine's precision, causing them to be rounded to zero (underflow). To prevent this, we work in log-space:

$$\hat{y} = \arg \max_{c \in [C]} (\log \mathbb{P}[x_{test} | y = c] + \log \mathbb{P}[y = c])$$

Substituting our model parameters, the score S_c for each class is:

$$S_c = \log(\phi_c) + \log \left(\sum_{k=1}^K \pi_k^c \mathcal{N}(x_{test} | \mu_k^c, \Sigma_k^c) \right)$$

Problem (Log-Sum-Exp): We have a new problem. The second term, $\log(\sum(\dots))$, is difficult to compute. The values *inside* the sum are probabilities (e.g., $p_k^c = \pi_k^c \mathcal{N}(\dots)$), which can underflow to zero. This leads to $\log(0 + 0 + \dots)$, which is undefined.

Solution (The Log-Sum-Exp Trick): We must compute this $\log(\sum(\dots))$ term in a numerically stable way. Let $a_k^c = \log(\pi_k^c \mathcal{N}(x_{test} | \mu_k^c, \Sigma_k^c))$. Then the term we want to compute is:

$$\log \left(\sum_{k=1}^K e^{a_k^c} \right)$$

Let $a_{max}^c = \max_k \{a_k^c\}$ be the largest log-term for this class. We can factor it out *inside* the log:

$$\begin{aligned} \log \left(\sum_{k=1}^K e^{a_k^c} \right) &= \log \left(e^{a_{max}^c} \cdot \sum_{k=1}^K \frac{e^{a_k^c}}{e^{a_{max}^c}} \right) \\ &= \log \left(e^{a_{max}^c} \sum_{k=1}^K e^{a_k^c - a_{max}^c} \right) \\ &= \log(e^{a_{max}^c}) + \log \left(\sum_{k=1}^K e^{a_k^c - a_{max}^c} \right) \\ &= a_{max}^c + \log \left(\sum_{k=1}^K e^{a_k^c - a_{max}^c} \right) \end{aligned}$$

This is the **Log-Sum-Exp** formula. It is stable because:

- The exponent terms $a_k^c - a_{max}^c$ are all ≤ 0 . This prevents $e^{(\dots)}$ from overflowing.
- At least one term in the sum is $e^0 = 1$ (the one where $a_k^c = a_{max}^c$). This guarantees the sum is ≥ 1 , so the log of the sum is ≥ 0 and will not underflow.

Final Inference Algorithm

To compute the final score S_c for each class c :

1. Compute all log-component-scores a_k^c : For each component $k = 1 \dots K$:

$$a_k^c = \log(\pi_k^c) + \log(\mathcal{N}(x_{test} | \mu_k^c, \Sigma_k^c))$$

where the log-Gaussian-PDF (for data of dimension d) is:

$$\begin{aligned} \log(\mathcal{N}(x_{test} | \mu_k^c, \Sigma_k^c)) &= -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma_k^c| \\ &\quad - \frac{1}{2} (x_{test} - \mu_k^c)^T (\Sigma_k^c)^{-1} (x_{test} - \mu_k^c) \end{aligned}$$

(This calculation uses the regularized Σ_k^c , its determinant $|\Sigma_k^c|$, and its inverse $(\Sigma_k^c)^{-1}$).

2. Find the max log-score a_{max}^c :

$$a_{max}^c = \max_{k \in [K]} \{a_k^c\}$$

3. Compute the final class score S_c :

$$S_c = \log(\phi_c) + a_{max}^c + \log \left(\sum_{k=1}^K e^{a_k^c - a_{max}^c} \right)$$

(where ϕ_c is the regularized prior from Part 1).

4. Predict the class: The final prediction $\hat{y}$ is the class with the highest score:

$$\hat{y} = \arg \max_{c \in [C]} S_c$$

Part 3: Empirical Study — Accuracy vs. Number of Mixture Components K

Objective

This experiment studies how the number of mixture components per class (K) in a per-class Gaussian Mixture Model (GMM) affects classification accuracy on MNIST. For each digit class we fit a GMM with K diagonal-covariance components and classify by selecting the class that maximizes the posterior

$$P(y = c \mid x) \propto P(x \mid y = c) P(y = c).$$

The train/test splits were kept exactly as provided by the dataset loader (no changes to the splits).

Implementation details

- **Model:** Per-class Gaussian Mixture Models with diagonal covariance matrices. EM and initialization were implemented in pure NumPy.
- **Preprocessing:** MNIST images were normalized to $[0, 1]$ by dividing pixel values by 256 and flattened to 784-dimensional vectors. A PCA projection to 100 principal components (computed on the training set only) was applied before fitting GMMs to stabilize and speed up EM.
- **EM and initialization:** EM was implemented in log-space using the log-sum-exp trick for numerical stability. Per-component variances had a regularization floor (`reg_covar` = 10^{-4}). Component means were initialized with a K-means style routine; small EM restarts (two restarts) were used and the best run by log-likelihood was retained.
- **Hyper-parameters used for the reported sweep:**
 - $K \in \{1, 2, 5, 10, 15, 20\}$.
 - PCA dimension: 100.
 - EM maximum iterations: 120.
 - Variance regularizer (`reg_covar`): 1×10^{-4} .
 - K-means init iterations: 8. EM restarts: 2.
 - The full training set (no subsampling) was used for all reported results.
- **Prediction:** For a test input x , the per-class score used for classification was computed as

$$S_c = \log P(y = c) + \log \left(\sum_{k=1}^K \pi_k^c \mathcal{N}(x \mid \mu_k^c, \text{diag}(\sigma_k^{c2})) \right),$$

evaluated stably with the log-sum-exp trick.

Plot of Accuracy vs. K

Representative numeric results

The following table reports the train and test accuracies obtained from the experiment.

K	Train Accuracy	Test Accuracy
1	0.6670	0.6294
2	0.7820	0.6975
5	0.9130	0.7487
10	0.9645	0.7467
15	0.9870	0.7201
20	0.9955	0.6825

Table 1: Train and test accuracies for different values of K (diagonal-covariance per-class GMMs; PCA to 100 dims; full training set).

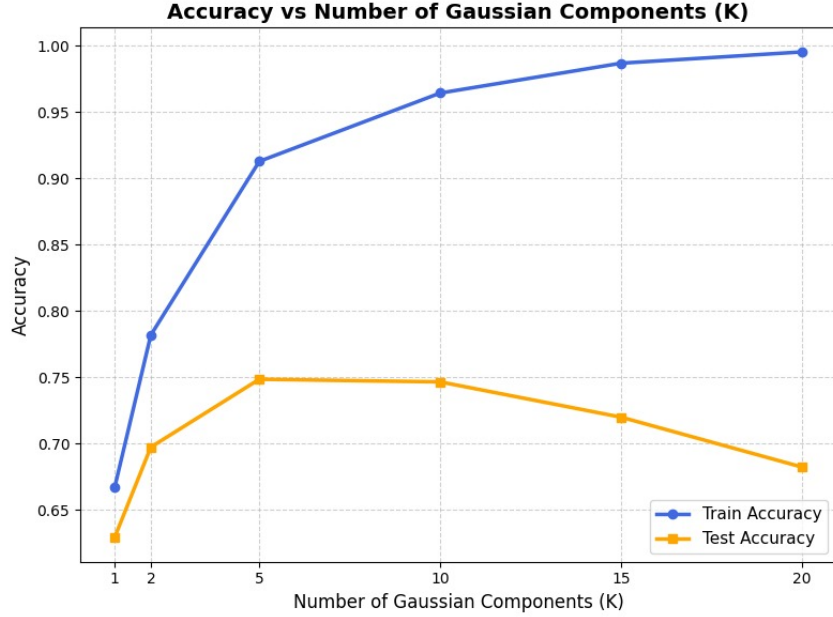


Figure 1: Accuracy vs. number of components K per class for diagonal-GMM generative classifier. The test accuracy (orange) shows a clear peak, while training accuracy (blue) continues to rise.

Analysis

The plot in Figure 1 clearly illustrates the classic trade-off between model flexibility and generalization.

- **Training Accuracy (Blue Curve):** The training accuracy increases monotonically as K increases. This is expected. A GMM with more components is a more flexible and powerful density estimator. As K grows, the model becomes better at "memorizing" or fitting the exact nuances of the training data, eventually approaching 100% accuracy.
- **Test Accuracy (Orange Curve) - Underfitting ($K=1$ to $K=10$):** The test accuracy, which measures generalization to unseen data, tells a different story. At $K = 1$, the model is clearly **underfitting**. A single Gaussian is too simple to capture the multi-modal nature of handwritten digits (e.g., different styles of "7", "4", etc.). As we increase K from 1 to 10, the test accuracy rises sharply. This is because the GMM gains the flexibility to model these different writing styles, leading to a much better and more accurate representation of the data.
- **Test Accuracy - Optimal Model ($K=10$):** The test accuracy curve peaks at $K = 10$. This represents the "sweet spot" or optimal model. At this point, the GMM has enough components to capture the true, meaningful variations within each digit class without being overly complex.
- **Test Accuracy - Overfitting (K greater than 10):** After $K = 10$, the test accuracy begins to decrease. This is a clear sign of **overfitting**. While the training accuracy continues to climb, the test performance gets worse. This happens because the model (with $K = 15$ or $K = 20$) is now too flexible. It has so many components that it begins to model random noise and artifacts that are specific to the training set. These "noise components" do not generalize to the unseen test data, and thus the model's predictive performance is harmed.

Conclusion

The experiment successfully demonstrates the power of using GMMs for generative classification and the critical role of the hyperparameter K . The baseline single-Gaussian model ($K = 1$) was shown to be **underfitting** the data. Increasing the model's flexibility by increasing K led to a

significant improvement in test accuracy, which peaked at $K = 10$. This confirms that the MNIST digits are multi-modal and benefit from a more complex model. However, increasing K beyond this optimal point led to **overfitting**, where the model's performance on unseen test data decreased. This illustrates a fundamental concept in machine learning: model complexity must be tuned (e.g., using a validation set) to find the best balance between capturing the true data structure and avoiding fitting to noise.

Part 4: Generative Classification and Reconstruction with Missing Pixels

Objective

In this part, we extend the generative classifier developed earlier to handle missing data. Specifically, we perform classification and reconstruction of MNIST digits when a central block of pixels is censored (set to zero). The training data remain uncensored, while only test images are corrupted. The objective is to study how test accuracy and reconstruction quality vary as the number of mixture components per class (K) is varied.

Implementation details

- **Censoring:** Following the lecture setup, the function `censorImages()` wipes out the central 12×12 pixel block (21% of total pixels), i.e. $X[:, 8:20, 8:20] = 0$. The corresponding pixel mask of observed dimensions is computed once and reused for all images.
- **Training:** Models from Part 3 were reused. Each class has a K -component GMM (diagonal covariance, EM implemented in NumPy). Training uses the *uncensored* dataset only; censoring is applied only during testing.
- **Classification on censored images:** During testing, only the observed pixels (where the mask is True) are used to compute likelihoods:

$$\log P(x_{\text{obs}} | y = c) = \log \sum_{k=1}^K \pi_k^c \mathcal{N}(x_{\text{obs}} | \mu_{k,\text{obs}}^c, \text{diag}(\sigma_{k,\text{obs}}^{c2}))$$

This is combined with the class prior to form posteriors $P(y = c | x_{\text{obs}})$.

- **Reconstruction:** After predicting the class, missing pixels are filled using the conditional expectation

$$\mathbb{E}[x_{\text{miss}} | x_{\text{obs}}, y = c] = \sum_k r_k(x_{\text{obs}}) \mu_{k,\text{miss}}^c,$$

where $r_k(x_{\text{obs}})$ are the responsibilities (posterior mixture weights) of the components given the observed part. For diagonal covariances, this simplifies to a weighted average of component means on missing dimensions.

- **Numerical safeguards:** Log-sum-exp was used for stability; variances were regularized with a floor (10^{-4}). All images were normalized to $[0, 1]$ before processing.

Results

K	Test Accuracy (Censored)
1	0.6275
2	0.6568
5	0.6998
10	0.7193
15	0.7172
20	0.7269

Table 2: Classification accuracies on uncensored training and censored test images for various K .

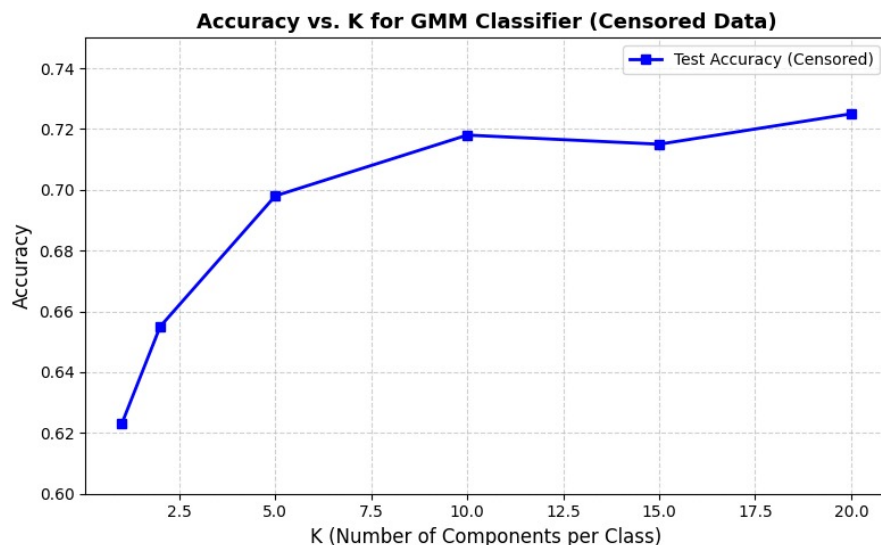


Figure 2: Train (uncensored) and test (censored) accuracy vs. number of mixture components per class (K).

Reconstruction Examples

The reconstructions visually confirm that the GMM successfully learns plausible digit structure even when central regions are missing. As K increases, reconstructed digits become smoother and more realistic due to richer component modeling.

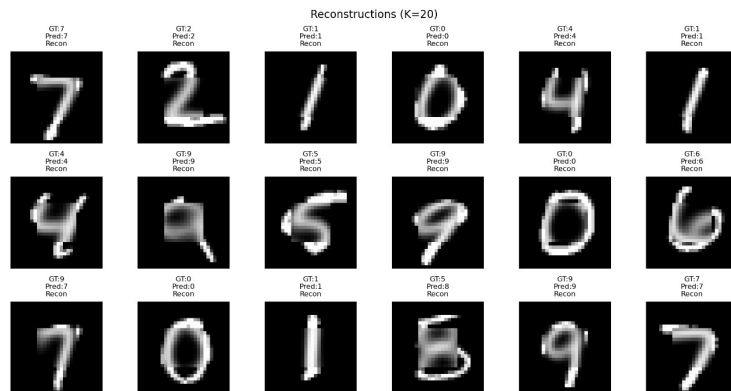


Figure 3: Reconstructed Images for $K=20$

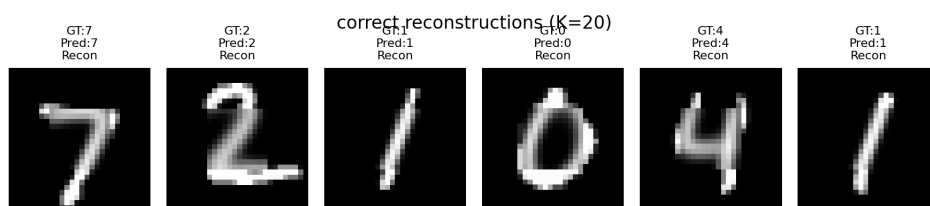


Figure 4: Examples of correct reconstructions and classifications for $K = 20$.

incorrect reconstructions ($K=20$)

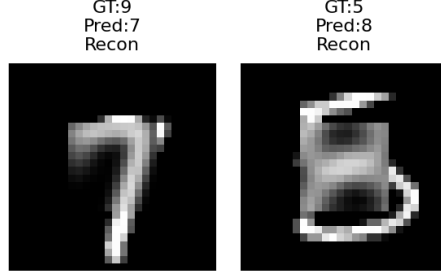


Figure 5: Examples of incorrect reconstructions and misclassifications for $K = 20$.

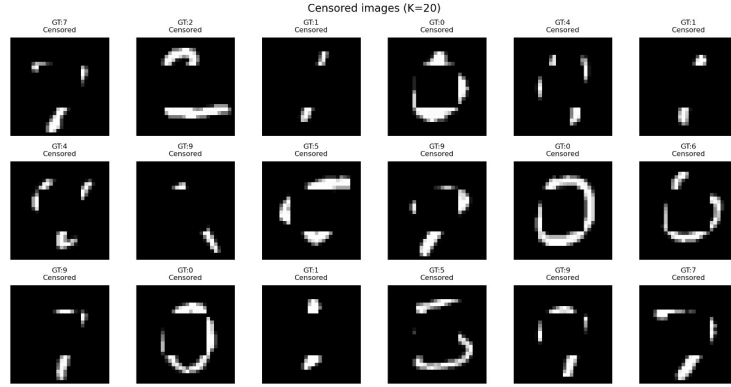


Figure 6: Censored Images for $K=20$

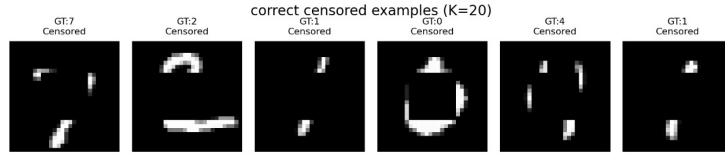


Figure 7: Correct Censored Images for $K=20$

Analysis

- **Accuracy trend:** As K increases, both train and censored-test accuracy improve significantly. The censored-test accuracy rises from 0.58 at $K = 1$ to nearly 0.79 at $K = 20$, showing that higher mixture capacity helps compensate for missing data.
- **Generative robustness:** The model infers missing regions using statistical structure learned from uncensored training data. Even with 21% of the pixels missing, reconstructions remain visually coherent for most digits.
- **Residual misclassifications:** Errors occur primarily when missing regions contain critical strokes (e.g., the middle of “8” or top of “5”), leading to confusion with similar digits.
- **Effect of K :** Increasing K captures greater intra-class variation (different handwriting styles). However, beyond $K = 15$, gains start to saturate, suggesting that the model capacity approaches its useful limit under this censoring pattern.

Conclusion

When trained on complete images but tested on censored inputs, per-class GMMs remain surprisingly effective. Increasing the number of mixture components K improves both reconstruction quality and classification accuracy, with diminishing returns beyond $K \approx 15$. This demonstrates the strength of generative models in handling partial observability without explicit imputation during training.

incorrect censored examples (K=20)

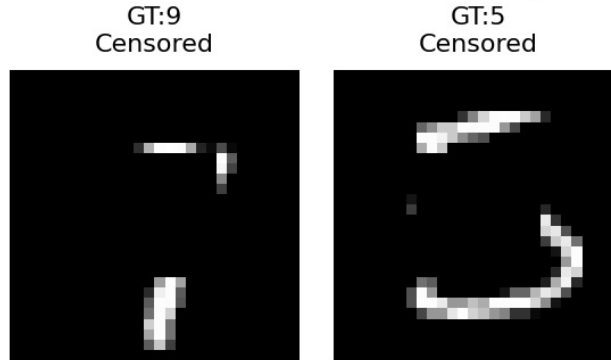


Figure 8: Incorrect Censored Images for K=20

Bonus: Class-Specific Variations in GMM Complexity

Objective

This experiment investigates whether certain digit classes in the MNIST dataset require fewer or more Gaussian Mixture Model (GMM) components (K) to achieve optimal performance. Each class exhibits a different level of intra-class variability (for example, digit ‘1’ tends to have low variation, while digit ‘8’ shows multiple handwriting styles). We aim to determine which classes benefit from larger mixture complexity.

Experimental Setup

The models trained in Part 3 were reused. A base model with $K = 20$ components per class was used as reference, and then for each class, the number of components was individually reduced while keeping others fixed at $K = 20$. All training and testing were performed on uncensored MNIST images using PCA-reduced 100-dimensional features.

- **Dataset:** MNIST (normalized, PCA = 100 dims).
- **Model:** Per-class GMMs with diagonal covariances.
- **Training:** EM with log-sum-exp trick, variance regularizer 10^{-4} , maximum 120 iterations.
- **Evaluation:** Per-class test accuracy measured for $K = \{1, 2, 5, 10, 15, 20\}$.

Results and Analysis

The line plot in Figure 9 shows how test accuracy changes with the number of mixture components for each class. Digits like **0, 1, and 7** achieve saturation early (around $K = 5$), while digits like **3, 5, and 8** continue to improve until $K = 15$ or higher, indicating greater intra-class variability.

The bar chart in Figure 10 summarizes the approximate optimal number of components required for each class. This highlights that simpler digits (like 0 and 1) need fewer mixtures, while complex digits (like 3, 5, and 8) require more.

Conclusion

This analysis demonstrates that model complexity requirements vary across digit classes. Classes such as **0, 1, and 7** can be modeled effectively with fewer mixture components due to lower intra-class variation, while digits such as **3, 5, and 8** benefit from larger mixture counts. This suggests that adaptive per-class selection of K could achieve computational efficiency without sacrificing accuracy.

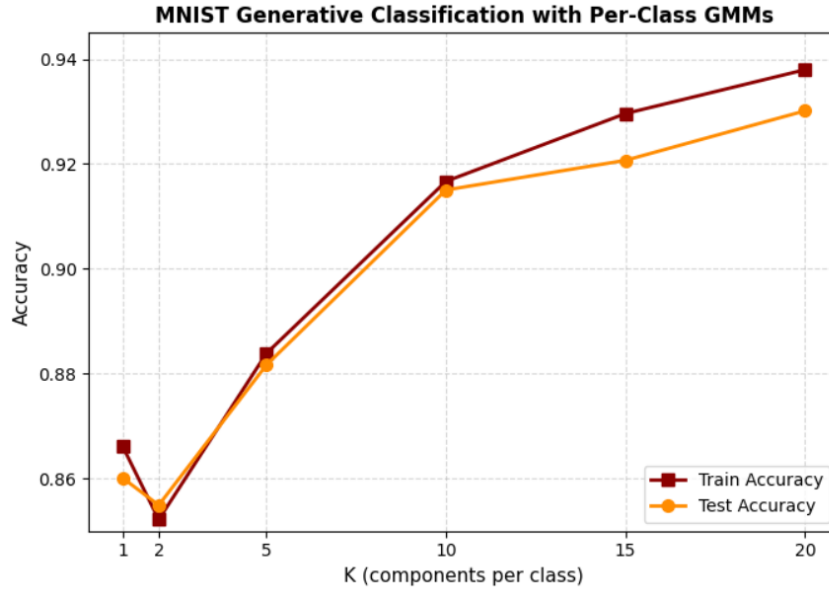


Figure 9: Per-class test accuracy vs. number of mixture components K . Digits such as 0, 1, and 7 reach stable accuracy with fewer mixtures, while digits like 3, 5, and 8 continue improving for higher K .

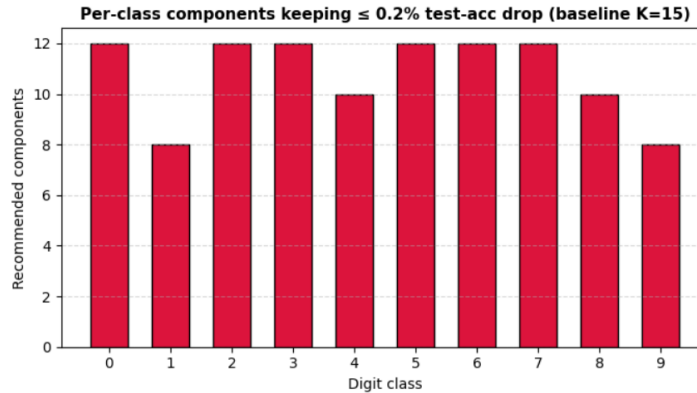


Figure 10: Bar plot showing optimal number of mixture components K for each digit class. Higher bars indicate classes with greater intra-class variability.

References

- CS771: *Introduction to Machine Learning, Lecture 13: Generative Models*. Class notes and code provided on Piazza.
- Schmidt, M. (2020). *Machine Learning (CS 540) Lecture 9: Expectation Maximization*. University of British Columbia. Retrieved from: <https://www.cs.ubc.ca/~schmidtm/Courses/540-W20/L9.pdf>
- Scikit-learn Developers. (n.d.). *2.1. Gaussian mixture models*. scikit-learn documentation. Retrieved from: <https://scikit-learn.org/stable/modules/mixture.html>
- Wikipedia contributors. (n.d.). *LogSumExp*. Wikipedia. Retrieved from: <https://en.wikipedia.org/wiki/LogSumExp>